

ADMINISTRATION SYSTÈME

NGINX PROXY MANAGER

Serveur Multimédia Privé Conteneurisé (Maquette Locale)



NGINX

PROXY MANAGER

Auteur	Kalandy (bthiery)
Programme	Auto-formation DevOps / SISR
Période	Avril 2026
Version	1.0.0 - Production
Environnement	Virtualisation locale (VirtualBox)

Partie I - Infrastructure Système (Socle)

Cette partie couvre la mise en place du socle système : provisionnement de la machine virtuelle, configuration réseau statique et durcissement SSH.

1.1 - Création de la VM (US-01)

Lancer Oracle VirtualBox et configurer la machine virtuelle avec les paramètres optimisés pour un serveur Docker léger.

➤ **Action : Dans VirtualBox, créer une nouvelle VM avec les paramètres ci-dessous.**

Nom	SRV-MEDIA-01
Type / Version	Linux / Debian (64-bit)
RAM	2048 Mo (2 Go)
Processeur	1 ou 2 vCPU
Disque dur	VDI dynamique - 20 Go

La convention de nommage SRV-MEDIA-01 est essentielle en administration système. Le ratio 2 Go de RAM est idéal pour faire tourner Docker, Jellyfin et un reverse proxy sans saturer le PC hôte.

1.2 - Configuration Réseau Statique (US-02)

L'objectif est de fixer l'adresse IP pour éviter que le Reverse Proxy ne perde le fil si le bail DHCP change.

1. Identifier l'interface réseau

➤ **Action : Dans le terminal Debian, taper `ip -c a` pour identifier le nom de l'interface réseau active.**

```
debian@debian:~$ ip -c a
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 ...
    inet 192.168.1.23/24 brd 192.168.1.255 scope global dynamic enp0s3
```

2. Configurer l'IP statique

➤ **Action : Éditer le fichier de configuration réseau : `sudo nano /etc/network/interfaces`**

```
# Remplacer la ligne dhcp par :
iface enp0s3 inet static
    address 192.168.1.23    # Choisir une IP libre du réseau
    netmask 255.255.255.0
    gateway 192.168.1.1    # IP de la box
    dns-nameservers 8.8.8.8 1.1.1.1
```

3. Appliquer et mettre à jour

➤ **Action : Relancer le réseau puis effectuer la mise à jour complète des paquets pour sécuriser le système.**
sudo systemctl restart networking

```
sudo ip addr flush dev enp0s3
sudo apt update && sudo apt upgrade -y
```

```
root@debian:~# ip -c a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:94:3d:a4 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.23/24 brd 192.168.1.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe94:3da4/64 scope link
        valid_lft forever preferred_lft forever
```

✓ **Résultat : Vérifier la prise en compte : ip -c a - L'interface enp0s3 doit afficher l'adresse statique 192.168.1.23/24.**

```
root@debian:~# ping -c 3 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=0.539 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=0.389 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=0.433 ms

--- 192.168.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2045ms
rtt min/avg/max/mdev = 0.389/0.453/0.539/0.062 ms
root@debian:~# ping -c 3 google.fr
PING google.fr (172.217.20.35) 56(84) bytes of data.
64 bytes from arnlls01-in-f3.1e100.net (172.217.20.35): icmp_seq=1 ttl=116 time=4.95 ms
64 bytes from arnlls01-in-f3.1e100.net (172.217.20.35): icmp_seq=2 ttl=116 time=7.50 ms
64 bytes from arnlls01-in-f3.1e100.net (172.217.20.35): icmp_seq=3 ttl=116 time=6.27 ms

--- google.fr ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 4.954/6.240/7.496/1.037 ms
```

✓ **Résultat : Tests de connectivité (Recette T-02) : ping -c 3 192.168.1.1 (passerelle) et ping -c 3 google.fr (résolution DNS). 0% de perte = succès.**

1.3 - Hardening SSH - Sécurisation de l'accès (US-03)

Le service SSH est souvent la première cible des attaques par force brute. Des mesures de durcissement inspirées des standards de sécurité applicative sont appliquées.

1. Génération de la paire de clés (depuis le PC Windows)

➤ **Action : Ouvrir un terminal PowerShell ou CMD sur le PC Windows et générer la paire de clés.**

```
ssh-keygen -t ed25519
# Appuyer sur Entrée pour l'emplacement par défaut.
# Définir une passphrase pour une sécurité maximale. (ex : debian)
```

2. Copie de la clé publique sur la VM

➤ **Action : Depuis le PC Windows, envoyer la clé publique vers la VM.**

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub debian@192.168.1.23
```

Si ssh-copy-id indisponible sur Windows, copier manuellement id_ed25519.pub dans
~/.ssh/authorized_keys sur la VM.

La clé se trouve dans : C:\Users\Utilisateur\.ssh\

3. Configuration du démon SSH sur Debian

➤ **Action : Éditer le fichier de configuration SSH : sudo nano /etc/ssh/sshd_config**

Modifications critiques à appliquer :

- Port 2222 - Changer le port 22 limite les scans automatiques.
- PermitRootLogin no - Interdire la connexion en tant que root.
- PasswordAuthentication no - Forcer l'utilisation des clés SSH.

```
GNU nano 2.9.3 /etc/ssh/sshd_config
# This is the sshd server system-wide configuration file. See
# sshd_config(8) for more information.

# This sshd was compiled with PATH=/usr/local/bin:/usr/bin:/bin:/usr/games

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented. Uncommented options override the
# default value.

Include /etc/ssh/sshd_config.d/*.conf

Port 2222
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
#PermitRootLogin no
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none

#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody

# For this to work you will also need host keys in /etc/ssh/ssh_known_hosts
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
# HostbasedAuthentication
#IgnoreUserKnownHosts no
# Don't read the user's ~/.rhosts and ~/.shosts files
#IgnoreRhosts yes

# To disable tunneled clear text passwords, change to no here!
PasswordAuthentication no
```

4. Application des changements

➤ **Action : Redémarrer le service SSH : `sudo systemctl restart ssh`**

⚠ **Attention : Désormais, préciser le port pour se connecter : `ssh -p 2222 debian@192.168.1.23`**

Partie II - Environnement Docker

Installation de Docker via le dépôt officiel pour garantir la stabilité et la portabilité de la stack.

2.1 - Installation de Docker Engine (US-04)

1. Mise à jour et installation des prérequis

➤ **Action : Mettre à jour les paquets et installer les dépendances.**

```
sudo apt update && sudo apt install -y ca-certificates curl gnupg
```

2. Ajout de la clé GPG officielle de Docker

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --
    dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

3. Ajout du dépôt Docker aux sources APT

```
echo \
    "deb [arch=$(dpkg --print-architecture) signed-
    by=/etc/apt/keyrings/docker.gpg]
    https://download.docker.com/linux/debian \
    $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
    sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

4. Installation du moteur et des plugins

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-
    plugin docker-compose-plugin
```

2.2 - Gestion des droits utilisateur (US-05)

Pour éviter d'utiliser sudo à chaque commande Docker, ajouter l'utilisateur au groupe docker.

➤ **Action : Ajouter l'utilisateur debian au groupe docker.**

```
sudo usermod -aG docker debian
```

⚠ Attention : *Se déconnecter (exit) et se reconnecter en SSH pour que le changement de groupe soit effectif.*

2.3 - Vérification de Docker Compose (US-06)

➤ **Action :** Vérifier que l'outil d'orchestration est bien présent.

```
docker compose version
```

```
debian@debian:~$ docker compose version
Docker Compose version v5.1.1
```

✓ **Résultat :** *La commande confirme le bon fonctionnement de l'orchestrateur en version v5.1.1.*

Partie III - Déploiement des Services (Infrastructure as Code)

Mise en place de l'arborescence, rédaction du fichier Docker Compose et déploiement de la stack.

3.1 - Structuration du répertoire de travail (US-07)

Création de l'arborescence dans le répertoire personnel de l'utilisateur pour garantir la persistance des données et la sécurité des droits.

➤ **Action :** Créer les dossiers de configuration et de données.

```
# Création des dossiers de configuration et de données
mkdir -p /home/debian/mediaserver/config/npm
mkdir -p /home/debian/mediaserver/config/jellyfin
mkdir -p /home/debian/mediaserver/data/media

# Positionnement dans le dossier de travail
cd /home/debian/mediaserver
```

3.2 - Fichier d'orchestration Docker Compose (US-08)

Création du fichier docker-compose.yml définissant les deux services npm et jellyfin sur un réseau isolé media-network.

➤ **Action :** Créer et éditer le fichier d'orchestration : nano docker-compose.yml

```
services:
  # US-08 : Reverse Proxy (Point d'entrée unique)
  npm:
    image: 'jc21/nginx-proxy-manager:latest'
    container_name: nginx-proxy-manager
    restart: unless-stopped
    ports:
      - '80:80' # Port HTTP
      - '81:81' # Interface Admin NPM
      - '443:443' # Port HTTPS
    volumes:
      - ./config/npm:/data
      - ./config/npm/letsencrypt:/etc/letsencrypt
    networks:
      - media-network

  # US-09 : Serveur Multimédia
  jellyfin:
    image: jellyfin/jellyfin:latest
    container_name: jellyfin
    user: 1000:1000 # UID/GID de l'utilisateur debian
    restart: unless-stopped
    volumes:
      - ./config/jellyfin:/config
      - ./data/media:/media
    networks:
      - media-network

networks:
  media-network:
    driver: bridge
```

```
services:
  npm:
    image: "jc21/nginx-proxy-manager:latest"
    container_name: nginx-proxy-manager
    restart: unless-stopped
    ports:
      - "80:80" # Port HTTP
      - "81:81" # Interface Admin NPM
      - "443:443" # Port HTTPS
    volumes:
      - ./config/npm:/data
      - ./config/npm/letsencrypt:/etc/letsencrypt
    networks:
      - media-network

  jellyfin:
    image: jellyfin/jellyfin:latest
    container_name: jellyfin
    user: 1000:1000 # UID/GID de l'utilisateur debian
    restart: unless-stopped
    volumes:
      - ./config/jellyfin:/config
      - ./data/media:/media
    networks:
      - media-network

networks:
  media-network:
    driver: bridge
```


3.3 - Déploiement et Recette (US-09)

➤ Action : Lancer la stack en arrière-plan : docker compose up -d

```
docker compose up -d
```

```
debian@debian:~/mediaserver$ docker compose up -d
[+] up 40/45
* Image jc2l/nginx-proxy-manager:latest [#####] 316.7MB / 385.4MB Pulling 15.1s
* Image jellyfin/jellyfin:latest [#####] 565.4MB / 691.8MB Pulling 15.1s

debian@debian:~/mediaserver$ docker compose up -d
[+] up 48/48
* Image jc2l/nginx-proxy-manager:latest Pulled 28.2s
* Image jellyfin/jellyfin:latest Pulled 32.0s
* Network mediaserver media-network Created 0.0s
* Container nginx-proxy-manager Started 0.6s
* Container jellyfin Started 0.6s
```

```
docker ps
```

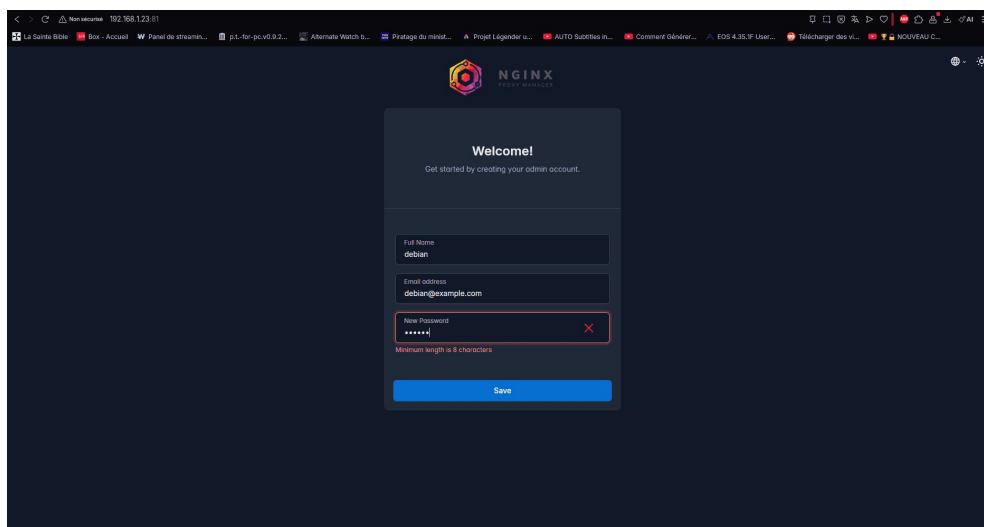
```
debian@debian:~/mediaserver$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
RTS
6dc3322d32d4   jc2l/nginx-proxy-manager:latest     "/init"                 37 seconds ago Up 36 seconds 0.0.0.0:80-81->80-81/tcp, [::]:80-81->80-81/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp
33984f991380   jellyfin/jellyfin:latest           "/jellyfin/jellyfin"    37 seconds ago Up 36 seconds (healthy) 8096/tcp
```

✓ Résultat : Les deux services affichent l'état Up (healthy). Jellyfin est prêt sur le port interne 8096, NPM sur les ports 80/81/443.

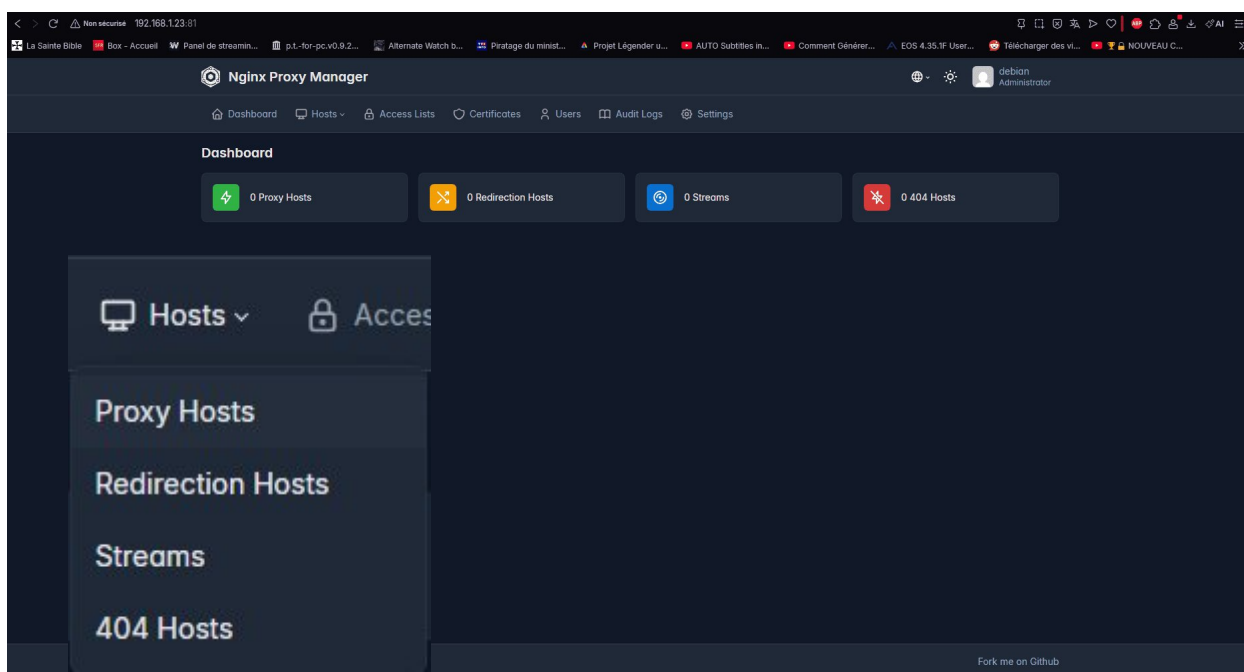
Partie IV - Exploitation & Configuration

4.1 - Configuration du Reverse Proxy (NPM)

Étape cruciale : elle permet d'accéder à Jellyfin via le port standard 80 géré par NPM, plutôt que par le port technique 8096.



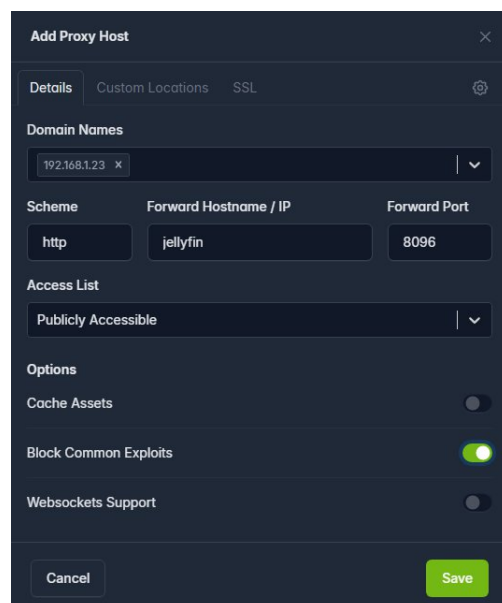
- Action : Écran 1 - Se connecter à l'interface NPM : <http://192.168.1.23:81> (Login : debian / Mdp : changeme). Configurer le compte administrateur.



- Action : Écran 2 - Dans l'onglet Hosts > Proxy Hosts, cliquer sur Add Proxy Host et renseigner les paramètres.

Paramètres de configuration du Proxy Host :

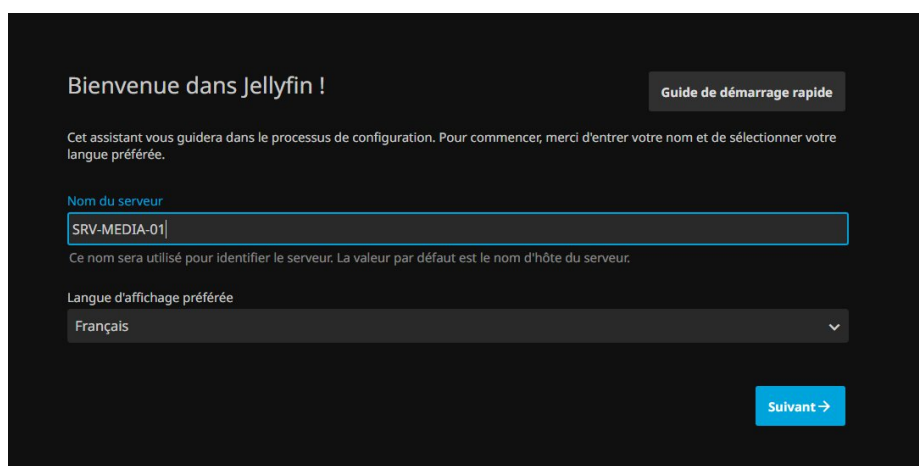
- Domain Names : 192.168.1.23 ou un nom local type media.lan
- Scheme : http
- Forward Hostname / IP : jellyfin (nom du conteneur Docker sur media-network)
- Forward Port : 8096 (port par défaut de Jellyfin)
- Activer Block Common Exploits pour la sécurité



4.2 - Initialisation de Jellyfin

Accès direct via http://192.168.1.23 (sans préciser de port). L'assistant de configuration de Jellyfin s'affiche.

1. Langue et Nom du serveur

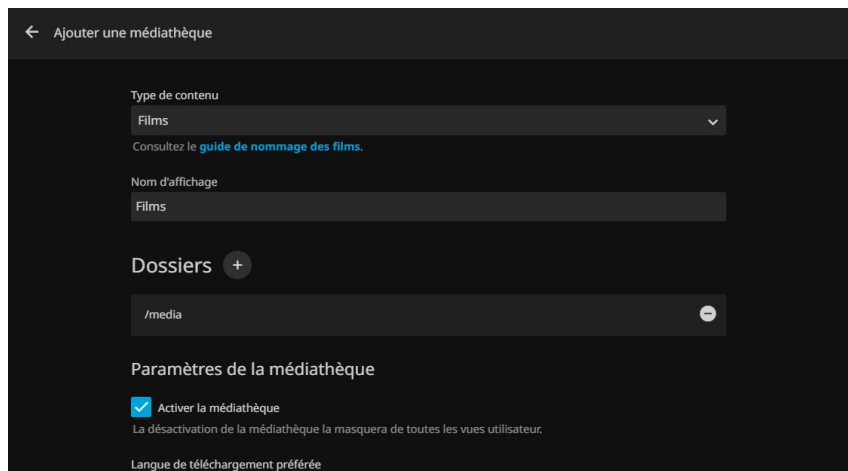


- Langue d'affichage préférée : (Français)
- Server Name : SRV-MEDIA-01 (cohérent avec la convention de nommage de l'infrastructure)

2. Création du compte administrateur

- Username : debian (cohérence avec l'utilisateur système) ou un pseudo de ton choix
- Password : Choisir un mot de passe robuste - le serveur est exposé via un proxy

3. Configuration des bibliothèques

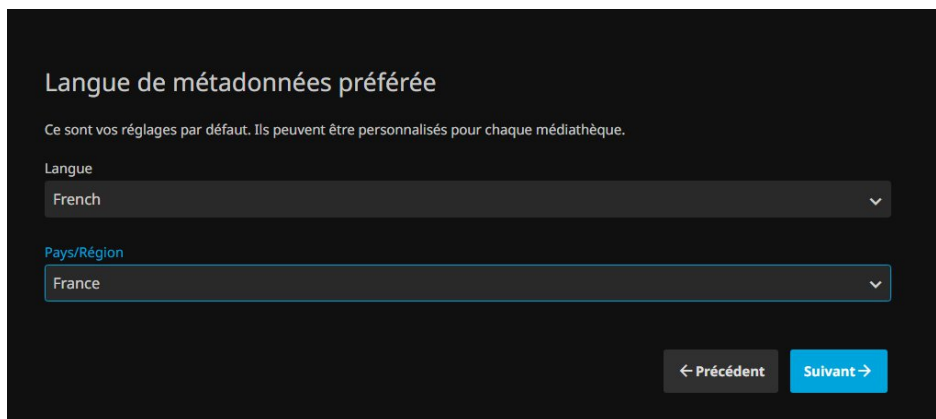


Lors de l'ajout de la médiathèque, le volume Docker relie automatiquement l'interface Jellyfin au stockage physique.

1. Cliquer sur Add Media Library et sélectionner le type de contenu (Films ou Séries).
2. Pour le dossier (Folders), cliquer sur + et sélectionner le chemin /media.

Le dossier /media dans l'interface correspond physiquement à /home/debian/mediaserver/data/media sur la VM Debian, grâce au volume configuré dans docker-compose.yml.

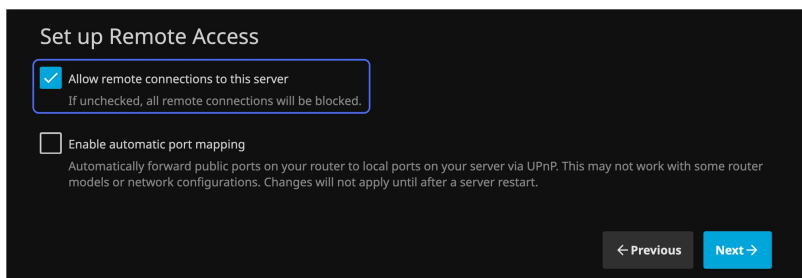
4. Récupération des métadonnées



- Langue : French
- Pays/Région : France

Jellyfin ira chercher automatiquement les affiches, résumés et notes en français.

5. Accès à distance



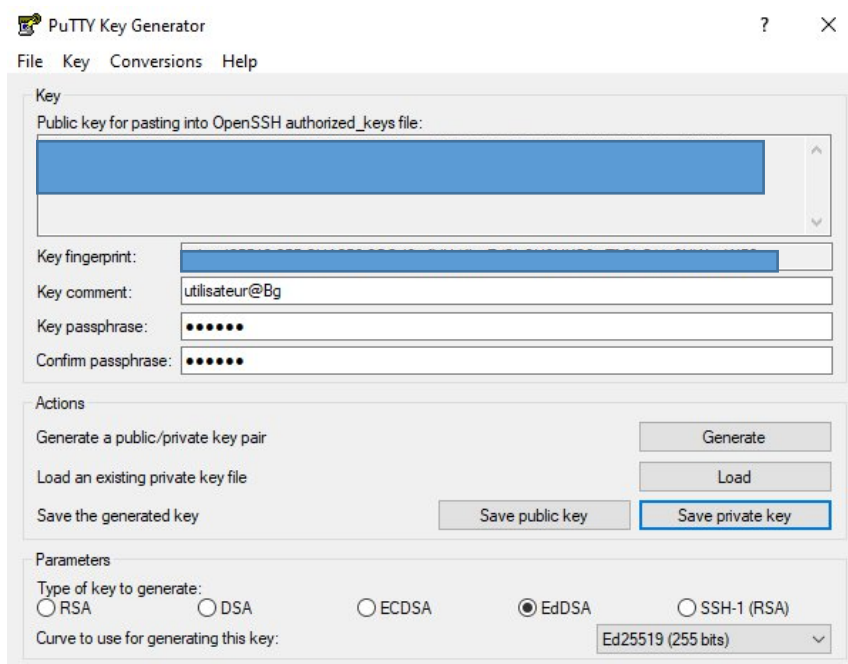
- Cocher Allow remote connections to this server.
- Ne pas activer le Port forwarding automatique (UPnP) - Nginx Proxy Manager gère déjà l'exposition de manière sécurisée.

4.3 - Injection du Premier Média - Validation T-03

Vérification que le pont entre le dossier physique Debian et le conteneur Jellyfin fonctionne réellement.

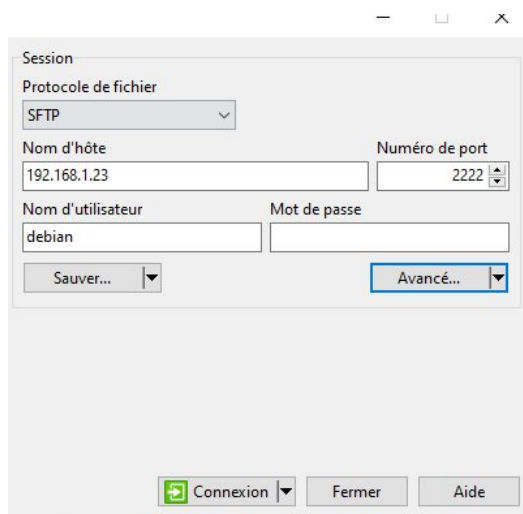
1. Convertir la clé privée pour WinSCP (format .ppk)

WinSCP et PuTTY utilisent le format propriétaire .ppk. La clé générée par ssh-keygen doit être convertie.



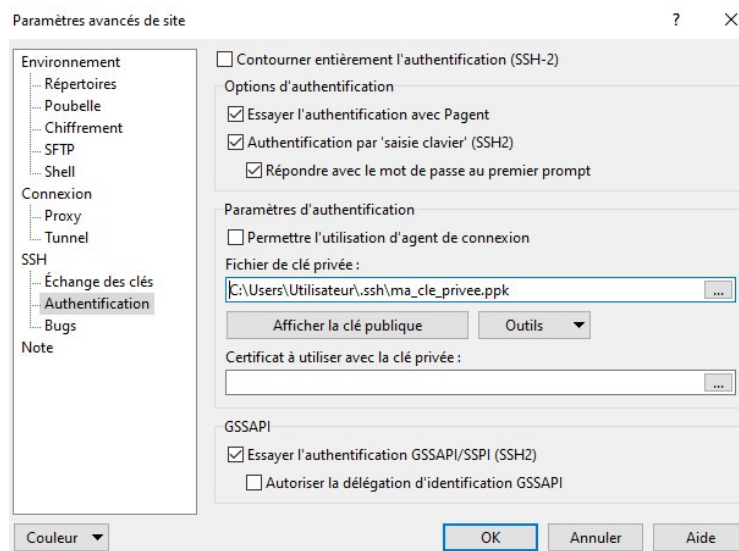
1. Lancer PuTTYgen sur Windows.
2. Cliquer sur File > Load private key.
3. Naviguer vers C:\Users\Utilisateur\.ssh\. Changer le filtre de fichiers sur "All Files (*.*)" pour voir id_ed25519 (sans extension).
4. Sélectionner id_ed25519 et cliquer sur Ouvrir. Saisir la passphrase (debian).
5. Cliquer sur Save private key et enregistrer sous ma_cle_privee.ppk.

2. Configurer WinSCP avec la clé .ppk



1. Dans WinSCP, aller dans Avancé... > SSH > Authentification.
2. Dans Fichier de clé privée, sélectionner ma_cle_privée.ppk.
3. Paramètres de connexion : Hôte 192.168.1.23 / Utilisateur debian / Port 2222.

3. Transfert du fichier et indexation



➤ **Action : Transférer un fichier vidéo (.mp4 ou .mkv) dans le répertoire : /home/debian/mediaserver/data/media/ sur la VM.**

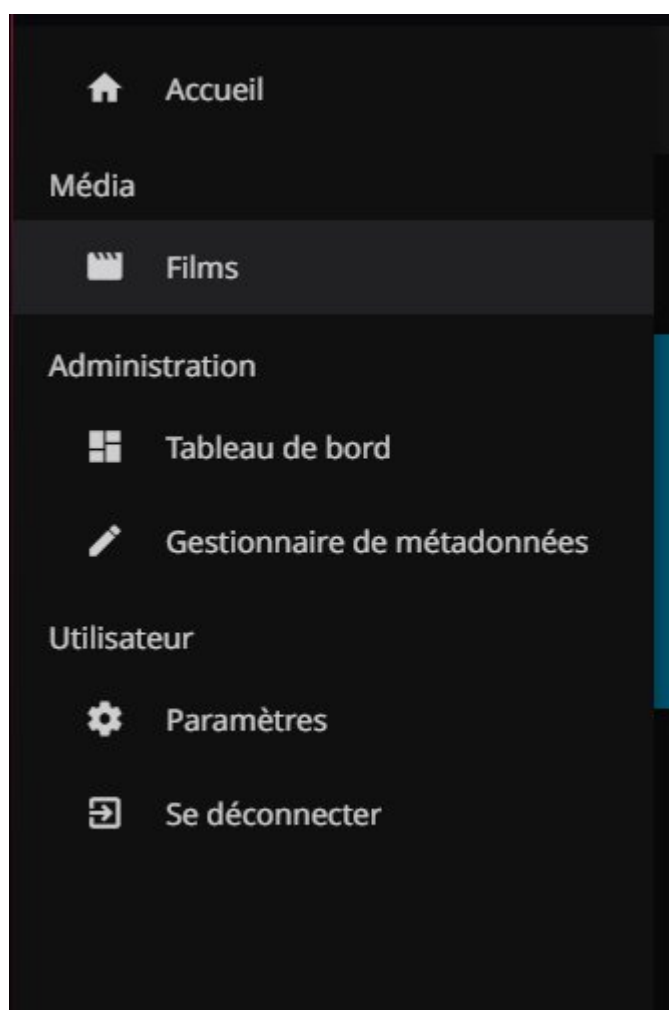
/home/debian/mediaserver/data/media/*.*					
Nom	Taille	Date de modification	Droits	Proprié...	
 Intro backrooms.mp4	6 081 KB	26/02/2026 17:12:55	rw-r--r--	debian	

✓ **Résultat :** Confirmation du transfert du fichier dans le répertoire cible de la VM.

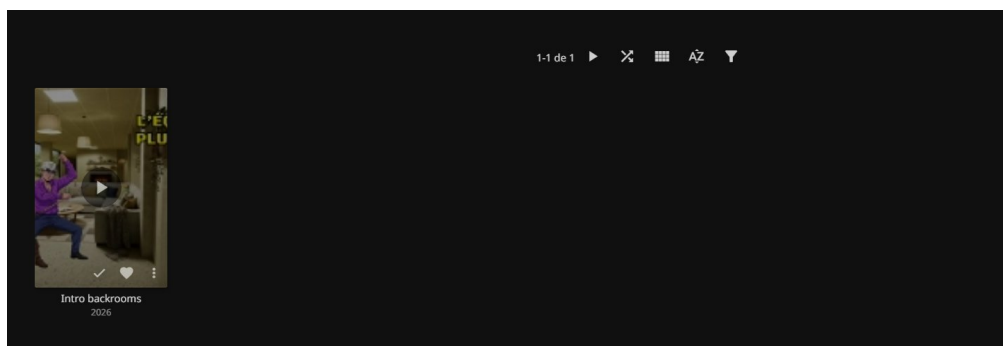
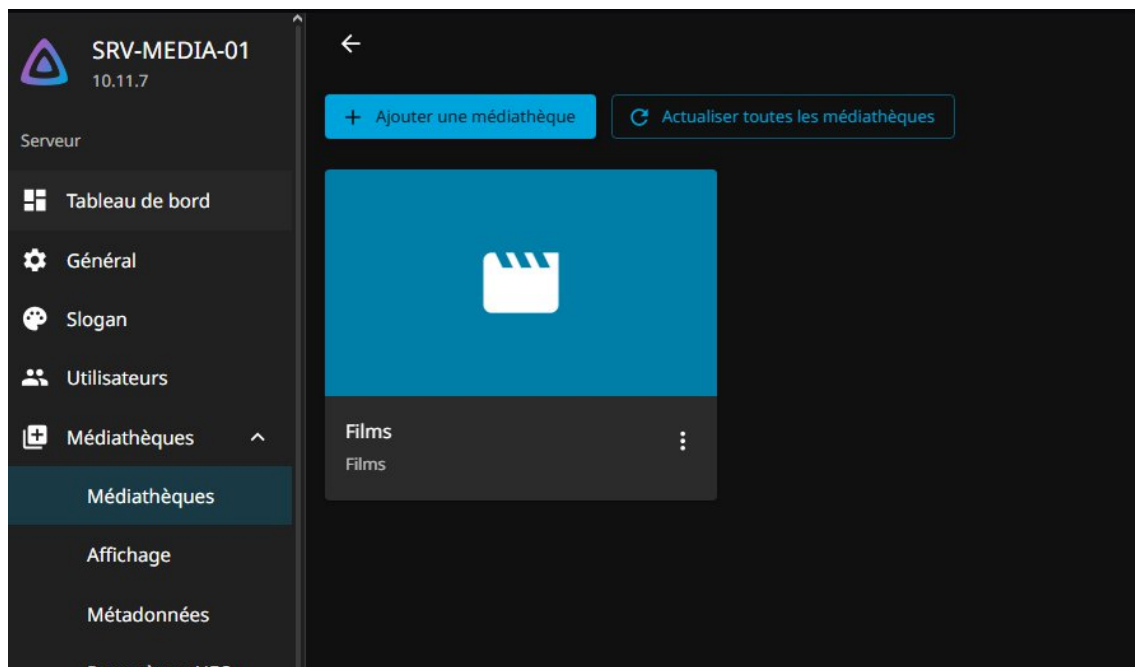
1. Retourner sur l'interface Jellyfin (<http://192.168.1.23>).
2. Aller dans Tableau de bord

Partie V - Résultat Final

Validation de l'infrastructure complète : Jellyfin indexe et lit correctement les médias transférés via SFTP.



➤ **Action : Menu d'administration Jellyfin permettant de rafraîchir les médiathèques manuellement.**



✓ **Résultat :** *Détail du menu contextuel permettant de gérer manuellement les métadonnées si nécessaire.*

Partie VI - Synthèse et Maintenance

6.1 - État Final du Backlog (Gestion de Projet)

L'intégralité des fonctionnalités prévues dans le périmètre initial a été livrée. Ce tableau confirme la complétion des objectifs techniques.

ID	Titre	Description	Statut
US-01	Provisionnement VM	Création de la machine Debian 12.	✓ Terminé
US-02	Configuration Réseau	Fixation de l'IP statique (192.168.1.23).	✓ Terminé
US-03	Hardening SSH	Port 2222 et authentification par clé .	✓ Terminé
US-04	Installation Docker	Déploiement du moteur Docker Engine.	✓ Terminé
US-05	Droits Utilisateur	Gestion du groupe docker pour l'utilisateur debian.	✓ Terminé
US-06	Docker Compose	Validation de l'outil d'orchestration v5.1.1.	✓ Terminé
US-07	Arborescence laC	Création des dossiers de persistance.	✓ Terminé
US-08	YAML de la stack	Rédaction du fichier docker-compose.yml.	✓ Terminé
US-09	Déploiement	Mise en production et configuration des services.	✓ Terminé

6.2 - Synthèse de la Sécurité Appliquée

L'infrastructure a été conçue selon le principe du moindre privilège et de la défense en profondeur.

Accès SSH : Sécurisé par clé Ed25519 sur le port 2222 ; accès par mot de passe et root interdits .

- Isolation Réseau : Les services communiquent via un réseau Docker isolé (media-network), non accessible directement depuis l'extérieur.
- Reverse Proxy : Nginx Proxy Manager sert de point d'entrée unique, masquant les ports techniques et filtrant les exploits communs.

6.3 - Guide d'Exploitation (Commandes de Maintenance)

Pour assurer la continuité de service, voici les commandes essentielles à utiliser dans le répertoire
~/mediaserver:

- Vérifier l'état des services :

```
docker compose ps
```

- Redémarrer la stack après modification :

```
docker compose up -d
```

- Consulter les logs de Jellyfin :

```
docker compose up -d
```

- Arrêter proprement le serveur :

```
docker compose down
```

Mission Conteneurisation complétée - Infrastructure Docker déployée, Reverse Proxy configuré, média indexé et accessible via Jellyfin.

Partie VII - Perspectives d'Évolution

Le projet "Serveur Multimédia" constitue une base évolutive pour une infrastructure plus complexe. Les étapes suivantes sont envisagées pour monter en compétences DevOps :

- Supervision (EPIC 4) : Intégration d'une stack Prometheus & Grafana pour monitorer l'utilisation des ressources (CPU, RAM, Température) en temps réel.
- Automatisation (EPIC 5) : Utilisation d'Ansible pour automatiser le déploiement de cette configuration sur n'importe quel nouveau serveur Linux.
- Accès Distant Sécurisé : Mise en place d'un tunnel WireGuard ou Cloudflare pour accéder aux médias hors du réseau local sans ouvrir de ports sur la box internet

— Fin de la documentation du projet —